NLP & ML for mental-health signals in social media

Personalized Adaptive Mental-Health Detection (PAMHED)

Progress Report 3

August 2025

Victor Ghattas (261180673)

Rayane Bouafia (261119286)

Abstract

Public social-media streams have enabled rapid advances in mental-health detection, but high in-domain scores often mask poor cross-platform generalizability, heuristic labeling, and limited transparency. Against this backdrop, we investigate personalized, behavior-based models for Twitter/X. The core hypothesis is that a user's deviation from their unique behavioral baseline is a more robust signal than post content alone. An end-to-end data pipeline was constructed to scrape user timelines, process media using BLIP, and engineer a rich set of behavioral features (temporal, emotional, linguistic). Initial experiments with a 4-class (control, depression, anxiety, bipolar) personalized "delta" model revealed a key finding: while the model excelled at identifying 'control' users with f1-score of 0.5779 by learning their "signal of normalcy," it struggled to differentiate between the mental health conditions due to overlapping signals of instability. Model explainability analysis using SHAP was crucial, uncovering that early models learned spurious shortcuts (e.g., from padding tokens) and confirming that the final model relied heavily on behavioral cues. This insight prompted a strategic pivot to a One-vs-Rest (OvR) binary classification strategy to isolate the unique signals for each condition. Reusable scripts for training these "expert" binary models and a framework for testing cross-platform generalizability on the eRisk 2018 Reddit dataset have been developed to validate the approach's robustness. A final pipeline was built to enable live, interpretable demos using SHAP, moving beyond "black box" predictions to explain why a user's behavior is flagged as indicative of a potential mental health concern.

Project Description

Purpose

The primary purpose of this project was to explore the value of personalization in computational mental health detection. The central scientific question was whether a model that analyzes a user's deviation from their own behavioral baseline could outperform a generic, text-only model. As the project progressed, the purpose evolved in response to initial findings. After discovering that a 4-class model struggled with the ambiguity of behavioral signals between different mental health conditions, the goal became more refined: to develop a more robust modeling strategy capable of untangling these overlapping signals. The current purpose is therefore to implement and evaluate a One-vs-Rest (OvR) ensemble of specialized binary classifiers. A secondary, but critical, purpose is to test the generalizability of the learned behavioral patterns by evaluating the Twitter-trained models on an out-of-domain Reddit dataset (eRisk 2018). Finally, the project aims to create a tool for explainable AI, providing not just predictions but interpretable, visual reasoning for those predictions for better understanding of how machine learning models work.

Description

The project was executed in three big steps under the workflow structure of three phases (detailed in the storyboards below):

- **Data Pipeline & Baseline Modeling:** This foundational step involved building a complete, end-to-end pipeline. Key components included a Python-based scraper for Twitter/X (using a Nitter backend), an image processing module that uses BLIP for rich captioning, and the core merge_blip.py script. This script is responsible for calculating per-user baselines and engineering dozens of advanced behavioral deviation features (temporal, emotional, linguistic, social, etc.) before exporting a training-ready Parquet file. This concluded with the training of the initial 4-class baseline (text-only mental-roberta-base) and personalized (delta) models.

- **Model Analysis & Strategy Pivot:** This step focused on deep analysis of the 4-class models (Experiment 1). We used the evaluate_models.py script and a visualization notebook (data_visual.ipynb) to assess performance, using matplotlib. Insight into personalization techniques exposed flaws in merge_blip.py and the data parquet. Explainability tools, specifically SHAP, were critical. They revealed that early models were learning from spurious data artifacts (like padding tokens) and confirmed that the final delta model's performance gains were driven by its ability to detect the "signal of normalcy" in control users, while it confused the "signals of instability" between the different mental health conditions. This data-driven insight led to a strategic pivot away from the single multi-class model.

- **(In progress) Advanced Binary Modeling & Generalization:** In response to the findings from experiment 1, this final step focuses on developing a more sophisticated One-vs-Rest (OvR) approach as part of experiment 2. This involves improving merge_blip.py as well as the current baseline and delta model scripts and from that, creating new, parameterized training scripts (train_binary_baseline.py, train_binary_delta.py) to build specialized "expert" models for each condition. Updated evaluation scripts will be built to compare models. In addition, to rigorously test cross-platform validity later down the line, a full set of scripts to parse, process, and evaluate the eRisk 2018 Reddit dataset was also created.

What was actually built

A comprehensive data processing pipeline script (merge_blip.py) that transforms raw tweet data into a feature-rich Parquet file.

Training scripts for both 4-class (train_delta.py) and parameterized binary (train_binary_delta.py), (train_binary_baseline.py) personalized models, along with their baseline counterparts (train_baseline.py) (train_binary_baseline.py).

A robust evaluation suite, including scripts to compare 4-class models (evaluate_models.py) and to compare binary models (evaluate_binary_models.py).

A Jupyter Notebook (data_visual.ipynb) for comprehensive performance analysis, including metric visualization and another notebook (SHAP.ipynb) for the generation of SHAP plots on random correctly predicted users for model explainability.

A complete, three-script pipeline (parse_erisk_xml.py, process_erisk_data.py, evaluate_on_erisk.py) specifically designed to test the cross-platform generalizability of the trained models on the eRisk 2018 Reddit dataset. Note that at this time, the pipeline has not been fully tested.

A live demo pipeline (testing_pipeline.py) capable of taking a new Twitter username, running the full data processing workflow, and feeding the result into the trained models for prediction and matplotlib analysis.

What does it do

The data pipeline automates the creation of a sophisticated dataset, calculating dozens of features representing each user's deviation from their established behavioral norms at the time of each tweet.
The training scripts produce machine learning models (both 4-class and binary) capable of classifying a tweet's associated mental state based on a combination of text and behavioral features.

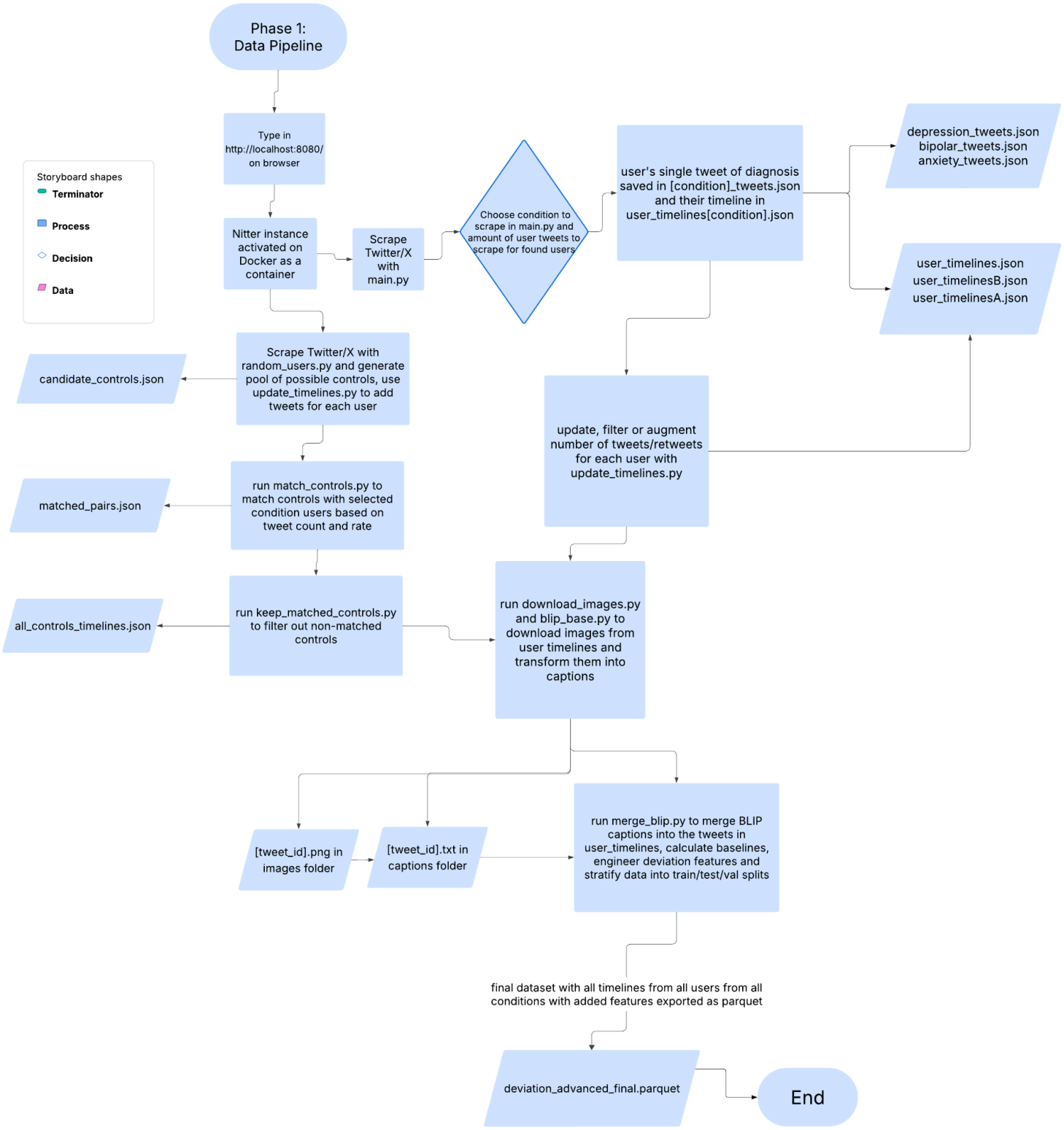
The evaluation scripts provide a robust framework for measuring and comparing model performance across different strategies, allowing for rigorous scientific validation.

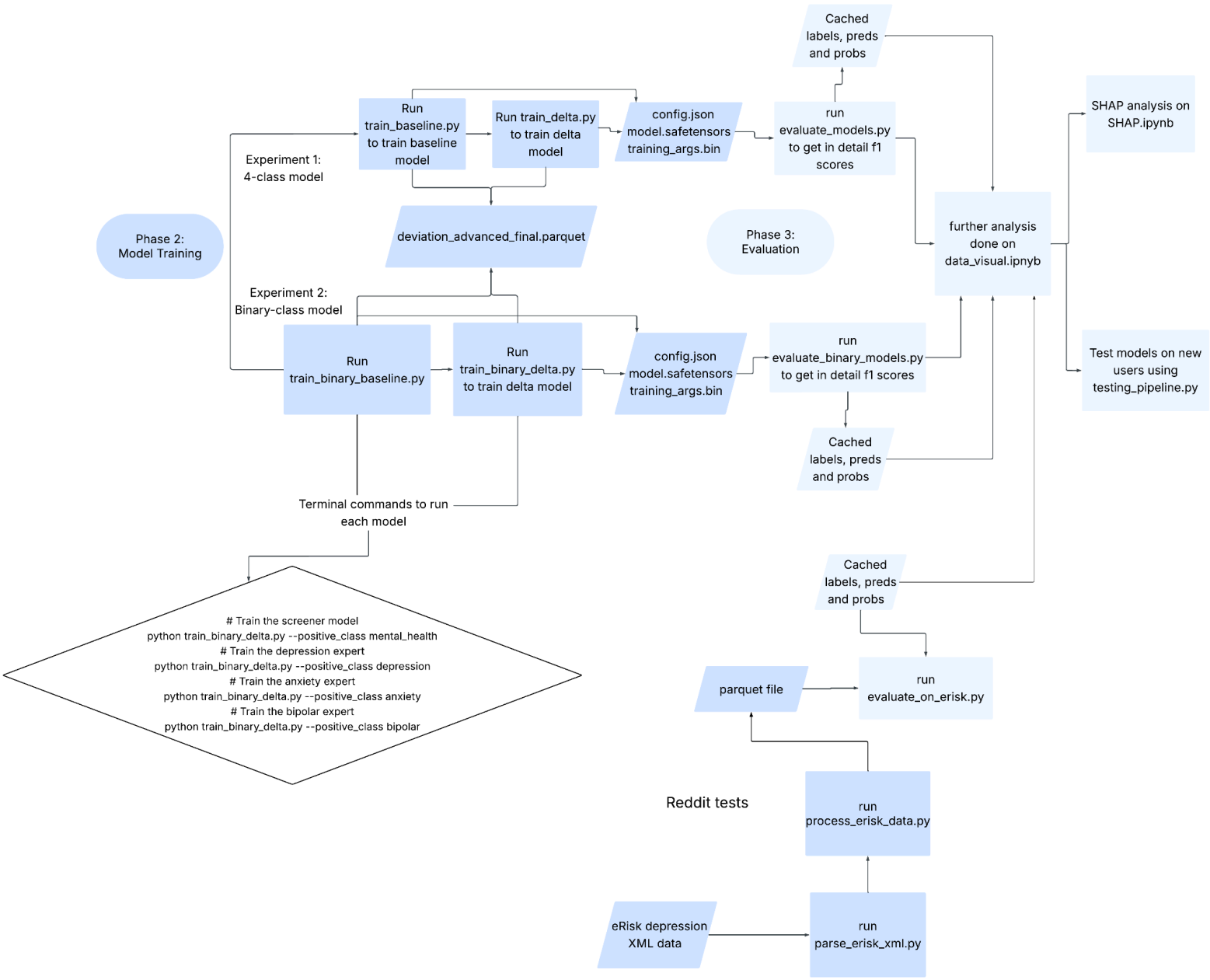
The cross-platform testing pipeline allows for the out-of-domain evaluation of the models, testing the core hypothesis that the learned behavioral signals are not specific to a single platform.

The live demo pipeline and SHAP analysis open up the "black box" of the model. For any given user, it can provide not just a prediction but a visual, interpretable explanation, highlighting the specific words and behavioral changes that led the model to its conclusion.

Detailed Project Description

Storyboard




Phase 2:
Model Training
Experiment 1:
4-class model
Run
train_baseline.py
to train baseline
model
Run train_delta.py
to train delta
model
config.json
model.safetensors
training_args.bin
run
evaluate_models.py
to get in detail f1
scores
Cached
labels, preds
and probs
deviation_advanced_final.parquet
Phase 3:
Evaluation
further analysis
done on
data_visual.ipnyb
SHAP analysis on
SHAP.ipynb
Experiment 2:
Binary-class model
Run
train_binary_baseline.py
Run
train_binary_delta.py
to train delta model
config.json
model.safetensors
training_args.bin
run
evaluate_binary_models.py
to get in detail f1 scores
Cached
labels, preds
and probs
Test models on new
users using
testing_pipeline.py
Terminal commands to run
each model
Train the screener model
python train_binary_delta.py --positive_class mental_health
Train the depression expert
python train_binary_delta.py --positive_class depression
Train the anxiety expert
python train_binary_delta.py --positive_class anxiety
Train the bipolar expert
python train_binary_delta.py --positive_class bipolar
Cached
labels, preds
and probs
parquet file
run
evaluate_on_erisk.py
Reddit tests
run
process_erisk_data.py
eRisk depression
XML data
run
parse_erisk_xml.py

Domain Model

Domain Model A: Data Processing & Feature Engineering

Key Domain Concepts (Entities)

Entity	Type	Fields / Properties	Constraints
UserTimeline	JSON file (from scraper)	username:str, tweets:list[{id, text, date, user, photos}]	One file per user; username unique
Image	Image file (JPG/PNG)	path:str, tweet_id:str,	One file per unique image URL
ImageCaption	Text file (.txt)	path:str, tweet_id:str,	One file per image
UserRecord	Row in master DataFrame	user_id:str, label:int, label_name:str, baseline_stats:dict	user_id unique; label is fixed
TweetObservation	Row in final Parquet file	tweet_id:str, user_id:str (FK), text:str, created_at:datetime, label:int, split:{train,val,test}, *plus all deviation feature columns*	tweet_id unique; must have associated UserRecord
Feature	Column in Parquet file	name:str (e.g., len_z, circadian_deviation), value:float	Calculated only for tweets after the baseline period

Relationships Between Entities

- A **UserTimeline** contains many tweets, each of which can have one or more **RawImages**.
- Each **RawImage** results in exactly one **ImageCaption** (1-to-1).
- All tweets from a **UserTimeline** are associated with exactly one **UserRecord** (many-to-1).
- A **UserRecord** is used to generate many **TweetObservations** (1-to-many).
- Each **TweetObservation** is enriched with many **DeviationFeatures** (1-to-many, represented as columns).

Domain Operations / Processes

Operation	Inputs	Outputs	Preconditions / Invariants
scrape_timeline	username	UserTimeline JSON file	Nitter instance is accessible; User exists and is public
caption	UserTimeline	Set of ImageCaption files	Raw images downloaded; BLIP model loaded
compute_features	UserTimeline, ImageCaptions, label	DataFrame of TweetObservations with DeviationFeatures	User must have > min_tweets; Timestamps are valid
add_splits	DataFrame of all users	DataFrame with a split column added	All users have been processed; label is assigned

Domain Model B: Model Training & Evaluation

Key Domain Concepts (Entities)

Entity	Type	Fields / Properties	Constraints
Dataset	Parquet file	path:str, schema containing all TweetObservation fields	Must be generated by Domain Model A
ModelDefinition	Python class (.py)	name:{Baseline, AdvancedDeltaModel}, base_model:str, hyperparameters:dict	Delta model requires feature engineering
TrainingRun	Directory on disk	path:str, model_type:{baseline, delta}, task:{multiclass, binary}, start_time, end_time	Each run has a unique output directory
Checkpoint	Directory on disk	path:str, config.json, model.safetensors, training_args.bin, epoch:int, f1_score:float	Saved periodically during a TrainingRun
EvaluationResult	Text/JSON output	metrics:dict (accuracy, F1, etc.), classification_report:str, confusion_matrix:array	Generated from a TrainingRun against test data
SHAPExplanation	Visualization (PNG)	shap_values:array, feature_names:list, tweet_text:str	Requires a trained model and a specific data instance

Relationships Between Entities

- The **ProcessedDataset** is the single input for many **TrainingRuns** (1-to-many).
- Each **TrainingRun** uses exactly one **ModelDefinition** (many-to-1).
- A **TrainingRun** produces many **Checkpoints** over its duration (1-to-many).
- A final **SCheckpoint** (the best one) from a **TrainingRun** is used to generate one **EvaluationResult** (1-to-1).
- A **Checkpoint** can be used to generate many **SHAPExplanations** (1-to-many).

Deployment Diagram or Deployment Description

A. Data Processing & Feature Engineering

Stack

Layer	Technology / Path
Host OS	Any (Windows, macOS, Linux)
Python venv	**../.venv/** isolated environment. pip install -r requirements.txt installs pandas, torch, transformers, ntscraper, nltk, etc.
Data Scraper	ntscraper (Python) pointed at a local or public Nitter instance (e.g., http://localhost:8080).
Image Captioner	transformers using Salesforce/blip-image-captioning-base.
Local Storage	**data/** directory structure for JSON timelines, images, captions, and final Parquet files. **pipeline_data/** directory for live testing on new user

Execution Flow

1. Run testing_pipeline.py with a username.
2. Scrape the timeline and download images.
3. Run BLIP on images to generate captions.
4. Run merge_blip logic to compute features.
5. Output a single-user Parquet file.

Orchestration

Manual CLI invocation. Can be wrapped to process a list of users sequentially.

B. Model Training & Evaluation

Stack

Layer	Technology / Path
Host OS / GPU	Any system with a CUDA-enabled NVIDIA GPU (e.g., local machine with RTX 3060).
Python venv	Same as above, plus scikit-learn, datasets, and safetensors.
Training Scripts	train_baseline.py, train_delta.py, train_binary_*.py and at the base for all these: Hugging Face Trainer API: mental/mental-roberta-base
Evaluation Scripts	evaluate_models.py, evaluate_binary_models.py, evaluate_on_erisk.py — load trained models and run inference.
Model & Data Storage	**data/** for the main Parquet dataset. **out/** for all training outputs, organized by run (e.g., **out/delta_final/**). cache/ for cached predictions and probabilities

Execution Flow

1. Login to huggingFace from CLI and enter token only after authenticating an account to access mental/mental-roberta-base model
2. Run a training script (e.g., train_delta.py)
3. Load the parquet dataset.
4. Train for a fixed number of epochs with early stopping.
5. Save the best model checkpoint to the corresponding out/ directory.
6. Evaluate models by running dedicated scripts, in these scripts, comment out if-else cache block in load_all() function only when evaluating models with a new dataset to update cached values

Orchestration

Manual CLI invocation for each training run.

Final Deliverables Table (history)

Developer	Due Date	Task	Points
Victor	May 2025	Brainstorming, psychology and machine learning, gaps in research	1
Victor and Rayane	May 30 2025	first report with project proposal due	1
Victor and Rayane	June 12 2025	Find definitive method of scraping twitter data and build objectives from there Reach out to other researchers for supplementary datasets	1
Victor and Rayane	June 19 2025	Build full scraping environment and scripts necessary for data collecting	2
Victor	June 30 2025	Data collection more than halfway finished Start of building of full pipeline	2
Victor	July 8 2025	Have dataset for one condition with controls ready and first training of models with that data	2
Victor	July 15 2025	Evaluation of models on jupyter notebook Midterm report and full pipeline demo	2
		Vacation	
Victor	July 29 2025	Full data collection finished Fine-tuning of models Research on extra delta features Revamp of code and training method for efficiency and better results	2
Victor	August 8 2025	Baseline and delta training done with full data	2
Victor	August 15 2025	Analyze metrics on notebook SHAP analysis Figure out possible improvements Final report and demo	1
Victor	August 25 2025	Binary classification models on top of multiclass models Robustness and cross-platform testing with reddit data Have clear results and performance comparisons for each model Start writing final paper	3

Information for Next Development Team

The project has a convenient but fragile data-acquisition path. The end-to-end pipeline works, but it depends on an unstable Nitter backend and on-the-fly processing, which limits large scale scraping and could break at any time. Consider the possibility of using free datasets or paying for X Research API to take this to the next level. There are a lot of issues regarding the model's performance so the focus should still be on improving the 2nd phase of this project (model training).

Highest-impact next steps

1. Debug and polish merge_blip script using data analysis in the notebooks to figure where baseline calculations and feature calculations could've gone wrong. Flaws in the logic were discovered as I was analyzing examples of user personalization in data_visual.ipynb

2. Train all four expert binary models to complete the OvR ensemble and look to fine-tune by increasing regularization and possibly clustering

3. Run cross-platform validation on the eRisk 2018 Reddit dataset to test generalizability of behavioral signals.

I will be continuing this project for the time coming but if someone else has to keep it going at some point, the execution steps outlined in the deployment description A and B, as well as the storyboards are crucial to understanding the workflow. With the groundwork done, the next team can move past plumbing and produce substantive findings on personalized behavioral modeling for mental health.

Experiment or Test Cases

Describe Experiment or Test Case

Experiment 1: 4-Class Baseline vs. Delta Model

Goal

To determine if the personalized delta model outperforms the text-only baseline model on a 4-class classification task (control, depression, anxiety, bipolar).

Methodology

1. **Base model — mental/mental-roberta-base.**
   Selected over generic roberta-base because it's pre-trained on >1.4B words from mental-health communities (mainly Reddit), giving it a head start on the tone, slang, and subtle cues common in this domain. This domain pretraining consistently outperforms generic pretraining for our task.

2. **Fine-tuning for 4 classes.**
   We attached a randomly initialized linear head and fine-tuned the *entire* network on our Twitter/X dataset to predict control, depression, anxiety, bipolar. Fine-tuning updates both the new head and the RoBERTa backbone so the rich base representations map cleanly to our labels.

3. **Handling class imbalance.**
   The training split was skewed toward **control**. To avoid majority bias, we computed class weights with sklearn.compute_class_weight('balanced') (weights $\propto$ 1 / class frequency) and pass them to CrossEntropyLoss, increasing the penalty for misclassifying minority classes.

4. **Expanding the set of features.**

   These features were calculated for every tweet after a user's initial baseline period and were globally standardized to ensure comparability across all users. A select_best_features() function will pick the top features for each user during delta training.

   Temporal rhythm & activity:

   - **Circadian deviation:** z-score of posting hour vs. user's baseline median hour. Higher = more atypical time.
   - **Late-night posting:** flag if tweet is in a sleep window (e.g., 23:00–04:00).
   - **Tweets-per-day deviation:** rolling 3/7/14/30-day TPD vs. baseline to capture short/long surges or dips.
   - **Bursts & silences:** flags for <1h since last tweet (burst) and >72h gap (silence).

   Emotional dynamics:

   - **Sentiment deviation:** z-score of VADER compound vs. user's baseline.

- **Emotional volatility:** standard dev. of sentiment over the last 10 tweets.
- **Polarity shift:** flag if sentiment flips sign relative to the previous tweet.

Linguistic & cognitive:

- **Length deviation:** z-score of characters per tweet vs. baseline.
- **Self-focus:** share of first-person pronouns (I/me/my).
- **Punctuation/caps:** counts of “!”/“?” and capitalization changes as energy/arousal cues.

Social engagement & anomalies:

- **Mentions:** number of @handles (proxy for interaction).
- **Isolation lexicon:** hits for words like “alone,” “lonely,” “nobody.”
- **Multivariate anomaly score:** sum of absolute z-scores across key signals (e.g., len_z, sentiment_z, circadian_dev) as a single “how abnormal” metric.

5. **Uncovering the blackbox**

   To understand the reasoning behind the models' predictions and diagnose weaknesses.

   The SHAP library was used to analyze individual predictions from the trained delta model, generating visualizations for both text and behavioral feature importance. A correct prediction on a depressed user was taken at random and analyzed using a SHAP bar plot and a waterfall plot on a jupyter notebook (SHAP.ipynb)

Training setup.
Baseline and personalized variants use the same regimen: 5 epochs, strong regularization, and early stopping, ensuring differences stem from modeling choices rather than training settings.

Before implemented weighted classes and advanced delta features (left) and after (right)

```
============================================================
BASELINE MODEL RESULTS:
============================================================
              precision    recall  f1-score   support

     control      0.496     0.758     0.600     65798
  depression      0.217     0.127     0.160     29215
     anxiety      0.267     0.181     0.216     30299
     bipolar      0.267     0.147     0.189     28668

    accuracy                          0.411    153980
   macro avg      0.312     0.303     0.291    153980
weighted avg      0.355     0.411     0.364    153980


============================================================
PERSONALIZED MODEL RESULTS:
============================================================
              precision    recall  f1-score   support

     control      0.500     0.744     0.598     65798
  depression      0.208     0.125     0.156     29215
     anxiety      0.262     0.176     0.211     30299
     bipolar      0.256     0.162     0.198     28668

    accuracy                          0.406    153980
   macro avg      0.306     0.302     0.291    153980
weighted avg      0.352     0.406     0.364    153980


============================================================
SUMMARY:
============================================================
Baseline Macro F1: 0.291
Personalized Macro F1: 0.291
Improvement: -0.000 (-0.2%)
```

```
======================================================================
BASELINE MODEL RESULTS (4-CLASS):
======================================================================
              precision    recall  f1-score   support

     control     0.6322    0.3770    0.4723     53894
  depression     0.2158    0.2490    0.2312     20853
     anxiety     0.2295    0.3199    0.2672     23939
     bipolar     0.2111    0.3054    0.2497     20401

    accuracy                         0.3309    119087
   macro avg     0.3222    0.3128    0.3051    119087
weighted avg     0.4062    0.3309    0.3507    119087


======================================================================
PERSONALIZED (DELTA) MODEL RESULTS (4-CLASS):
======================================================================
              precision    recall  f1-score   support

     control     0.5626    0.5940    0.5779     53894
  depression     0.2187    0.2095    0.2140     20853
     anxiety     0.2458    0.2341    0.2398     23939
     bipolar     0.2267    0.2156    0.2210     20401

    accuracy                         0.3895    119087
   macro avg     0.3134    0.3133    0.3132    119087
weighted avg     0.3811    0.3895    0.3851    119087


======================================================================
SUMMARY COMPARISON:
======================================================================
Metric                    Baseline        Delta     Improvement
----------------------------------------------------------------------
Accuracy                   0.3309       0.3895         +0.0587
Macro Precision            0.3222       0.3134         -0.0087
Macro Recall               0.3128       0.3133         +0.0005
Macro F1                   0.3051       0.3132         +0.0081
```

Results

The evaluation produced a clear and surprising result. The delta model did not improve on the detection of mental health conditions. Instead, it became a highly accurate detector for the control group, showing a +10.6% F1-score improvement for that class. However, its performance on Depression, Anxiety, and Bipolar slightly decreased (respectively from 0.23 to 0.21, 0.26 to 0.23, 0.24 to 0.22) This indicates that while behavioral features provide a strong "signal of normalcy," the signals of "instability" for the different mental health conditions are too noisy and overlapping for a multi-class model to easily differentiate.

Image 1

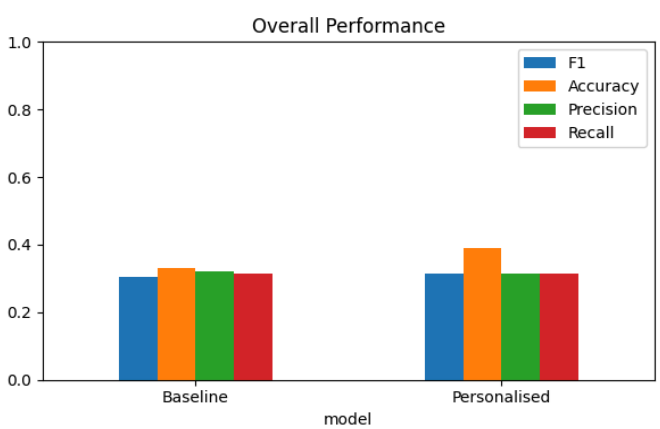


Image 2

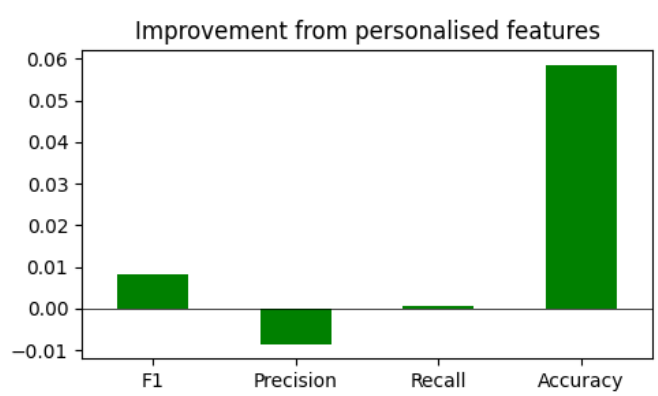


Image 3

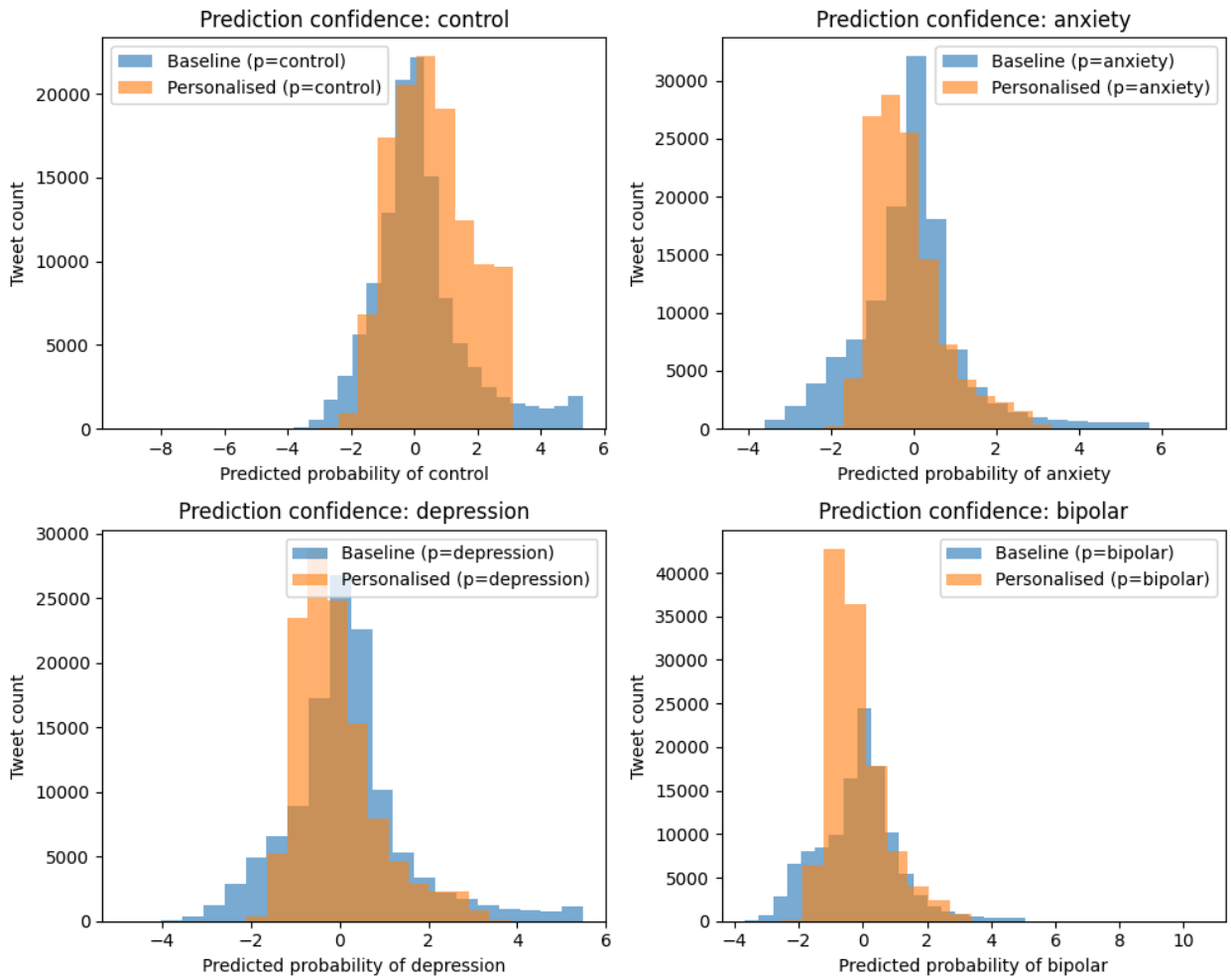
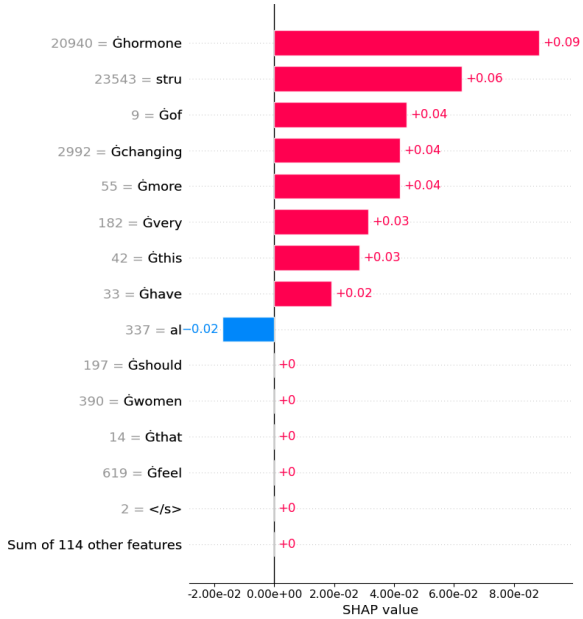


Image 4: Example 1 of tweet analyzed for SHAP

```
Searching for a tweet correctly predicted as 'depression'...
Found a good example to explain at test_df index: 68204

--- Example Tweet ---
If you have ever felt overwhelmed, tearful, anxious or even angry in the run up to your period, then you might have experienced premenstrual syndrome (PMS). Some women can also e
```

Image 5: Waterfall pot analysis of example 1



Ġ is just a token used to separate words, it's not part of the text

Image 6: Example 2 of tweet analyzed by SHAP

```
Searching for a tweet correctly predicted as 'depression'...
Found a good example to explain at test_df index: 38298

--- Example Tweet ---
I feel so lonely and sad. Like a hole in my chest. Like my heart is shrinking without ever stopping. And I feel tired. I keep drinking to be able to fall asleep
```

Image 7

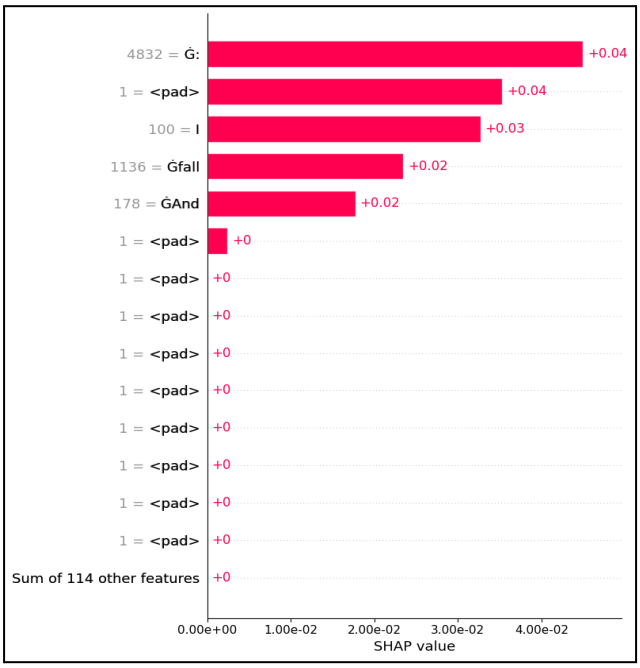
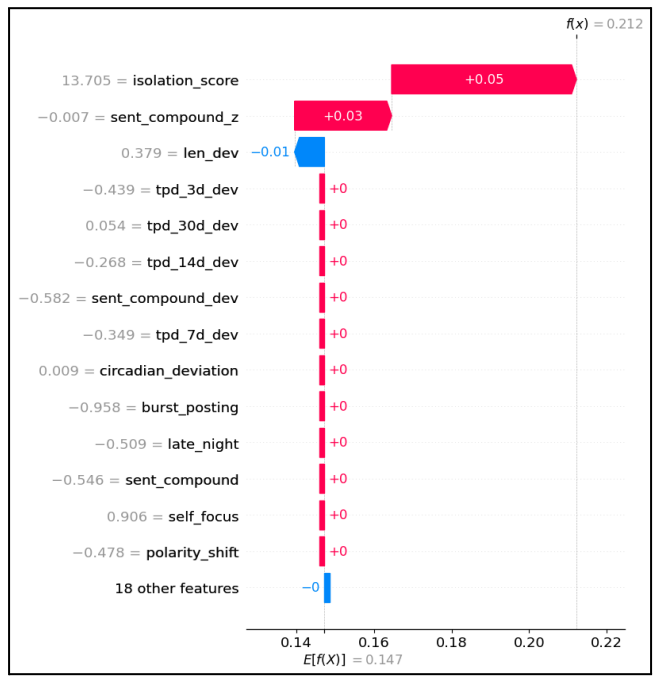


Image 8



Isolation score high due to words like lonely being used, which are rare from this user

A look at the logit distributions (Image 3) shows a consistent pattern across all four classes. The baseline (blue) has heavier tails and often pushes very high confidences, especially for anxiety, with many logits > 2. The personalized delta (orange) stays centralized and cautious, clustering near 0; for control it forms a sharp peak around 0 versus the baseline's wider spread. Net effect: the personalized model applies a more nuanced boundary, recognizing that high deviation doesn't equal pathology, and is less overconfident even when the final label doesn't change. This is continuous with the previous findings from the progress report 2.

An observed decrease in training loss, but an increase in evaluation loss during training also suggested overfitting, which explains the lack of predictive accuracy both models have, as seen in images 1 to 2

The SHAP plots in images 5, 7 and 8 were instrumental. They confirmed that behavioral features had a very small impact on predictions just like text features, which seemed to not be understood by the model. Critically, they revealed the model was learning spurious shortcuts, such as assigning high importance to `<pad>` tokens (used to pad shorter sentences to the max_length), which either could be mean the model favors shorter text when diagnosing a user or simply that it has not learned to properly distinguish between signal and noise. Despite the low differential scores, the plots demonstrated that the model was correctly identifying behavioral deviations (like burst posting and high emotional volatility) as key indicators.

Reflection

What would you do differently (what have you learned negative)

- We significantly underestimated the challenge of a 4-class problem where the signals for three of the classes are highly correlated and overlapping. We should have anticipated this "inter-class confusion" and perhaps started with a simpler binary problem.
- Initial Training Regularization: Our first training runs were under-regularized, which led to the model learning spurious correlations (like from padding tokens), a problem only discovered through the SHAP analysis. More aggressive regularization should have been the default from the start.
- We spent considerable time debugging and trying to improve the 4-class model when the results themselves were pointing towards a fundamental strategic issue. Pivoting earlier to the One-vs-Rest approach after the first evaluation would have been more efficient.
- Upon randomizing correctly predicted tweets, some were observed to be in a different language. A better filtration logic needs to be implemented to merge_blip.py for a better exported parquet

What would you do again (what have you learned positive)

- When using PyTorch, making sure I'm using CUDA to utilize my GPU for training and not just my CPU. Changing my training scripts, only recently, to maximize efficiency and updating to a CUDA-enabled PyTorch wheel saved me many hours
- Using nitter and ntscraper was a great way to get free access to large amounts of data, even with its issues, it allowed me to bypass the paid X research API program

Future Work

Things to Fix

- Upon randomizing correctly predicted tweets, some were observed to be in a different language. A better filtration logic needs to be implemented to merge_blip.py for a better exported parquet
- Upon analyzing baseline deviation it was found that anomaly scores were constantly similar amongst users showcasing perhaps another logic flaw to fix in merge_blip.py
- More robust sentiment estimator (e.g., distilRoBERTa-sentiment) to reduce flat deltas.
- Experiment 2: One-vs-Rest (OvR) Classification for depression detection:
    - Train baseline and delta binary classification models for depression vs others (Depression Expert)
- Refine Feature Selection for Binary Models:
    - The 'select_best_features' function should be run for each binary task (`Depression vs. Others`, etc.) to find the optimal feature set for each specific problem.

Features to Add

- Implement Cross-Platform Validation:
    - Execute the full eRisk pipeline: parse, process, and evaluate the Reddit data using the trained Twitter "Depression Expert" model to formally measure cross-platform generalizability.
- Finalize OvR Training:
    - Complete the training and evaluation of all four "expert" binary models, depression, bipolar, anxiety and non-control detection, to get a final performance metric for the approach, and compare it to the four binary baseline models
- Clusters based on different writing styles measured using LIWC categories

Suggestions

- Acquire a larger dataset but also more Labeled Data:
    - All deep learning models benefit from more data. Future efforts could focus on expanding the user base, particularly for the underrepresented 'anxiety' and 'bipolar' classes. This would improve the statistical power of the models and allow for the engineering of even more nuanced features.

References and Background Material (accumulated)

Aldkheel, A., & Zhou, L. (2024). Depression detection on social media: A classification framework and research challenges and opportunities. Journal of Healthcare Informatics Research, 8(1), 88–120. https://doi.org/10.1007/s41666-023-00152-3

Chancellor, S., & De Choudhury, M. (2020). Methods in predictive techniques for mental health status on social media: A critical review. NPJ Digital Medicine, 3(43). https://doi.org/10.1038/s41746-020-0233-7

Coppersmith, G., Dredze, M., Harman, C., Hollingshead, K., & Mitchell, M. (2015). CLPsych 2015 shared task: Depression and PTSD on Twitter. In Proceedings of the 2nd Workshop on Computational Linguistics and Clinical Psychology: From Linguistic Signal to Clinical Reality (pp. 31–39). Association for Computational Linguistics. https://aclanthology.org/W15-1204.pdf

Di Cara, N. H., Maggio, V., Davis, O. S. P., & Haworth, C. M. A. (2023). Methodologies for monitoring mental health on Twitter: Systematic review. Journal of Medical Internet Research, 25, e42734. https://doi.org/10.2196/42734

Harrigian, K., Aguirre, C., & Dredze, M. (2020). Do models of mental health based on social media data generalize? In Findings of the Association for Computational Linguistics: EMNLP 2020 (pp. 3774–3788). https://doi.org/10.18653/v1/2020.findings-emnlp.337

Losada, D. E., Crestani, F., & Parapar, J. (2018). Overview of eRisk 2018: Early risk prediction on the Internet. In Working Notes of CLEF 2018 – Conference and Labs of the Evaluation Forum (CEUR Workshop Proceedings, Vol. 2125). CEUR-WS.

Mármol-Romero, A. M., Moreno-Muñoz, A., Plaza-del Arco, F. M., Molina-González, M. D., Martín-Valdivia, M. T., Ureña-López, L. A., & Montejo-Ráez, A. (2024). Overview of MentalRiskES at IberLEF 2024: Early detection of mental disorders risk in Spanish. Procesamiento del Lenguaje Natural, 73, 435–448. https://ceur-ws.org/Vol-3756/MentalRiskES2024_paper6.pdf

Waleed Bin Tahir et al. Depression detection in social media: A comprehensive review of machine learning and Deep Learning Techniques. IEEE Journals & Magazine. https://ieeexplore.ieee.org/document/10843708/

Wu, Y., Liu, Z., Yuan, J. H., Chen, B., Cai, H., Zhu, L., … Hu, B. (2024). PIE: A personalized information embedded model for text-based depression detection. Information Processing & Management, 61(6), 103830. https://doi.org/10.1016/j.ipm.2024.103830

Sources for Advanced Deviation Features

Wang, W., et al. (2018). "Depression and Bipolar Disorder Detection on Social Media using a Deep-Learning-based Approach." *Neuropsychopharmacology.*

- *Relevance:* Directly links irregular daily posting patterns (circadian disruption) and changes in activity levels on social media to bipolar disorder and depression.

Hassan, F. B., et al. (2022). "Linguistic and Emotional Volatility in Bipolar Disorder Detection." *IEEE Access.*

- *Relevance:* Directly investigates and confirms that emotional volatility and linguistic variability in social media text are powerful features for discriminating between users with bipolar disorder and control subjects.

Eichstaedt, J. C., et al. (2018). "Facebook language predicts depression in medical records." *Proceedings of the National Academy of Sciences (PNAS).*

- *Relevance:* A large-scale study that validates the use of linguistic markers from social media (including those related to self-focus and negative emotions) to predict a clinical diagnosis of depression from patient medical records.

Torous, J., & Firth, J. (2018). "The Digital Phenotype." *Harvard Review of Psychiatry.*

- *Relevance:* This review article defines the concept of the "digital phenotype"—quantifying an individual's behavioral patterns from digital devices. It explicitly frames social interaction patterns (like mention counts) and deviations from normalcy (anomalies) as key components for understanding mental health.